# SMARTMART "Online Ordering & Delivery Platform"

**A Real-Time Research Project Report**

**Submitted in partial fulfillment of the requirements for the award of the degree of**

**BACHELOR OF TECHNOLOGY**

**In**

**COMPUTER SCIENCE AND ENGINEERING (AIML)**

**By**


| | |
|---|---|
| **CH. AKHIL** | **22VD1A6603** |
| **R. SUMANTH** | **22VD1A6647** |
| **A. SHIVAKUMAR** | **23VD5A6612** |

**Under the guidance of**

**G. SRIDHAR**

**Assistant Professor**

**Department of Computer Science and Engineering.**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AIML)**

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD**

**UNIVERSITY COLLEGE OF ENGINEERING MANTHANI**

Centenary Colony, Pannur (Vill), Ramagiri (Mdl), Peddapalli-505212, Telangana (India).


2023-2024

# JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
# UNIVERSITY COLLEGE OF ENGINEERING MANTHANI
# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AIML)

## DECLARATION BY THE CANDIDATE

We,

| | |
|---|---|
| **CH. AKHIL** | **22VD1A6603** |
| **R. SUMANTH** | **22VD1A6647** |
| **A. SHIVAKUMAR** | **23VD5A6612** |

hereby declare that the project report entitled **smart-mart "online ordering & delivery platform"** under the guidance of **Mr.G.Sridhar, Assistant Professor, Department of Computer Science and Engineering, Jntuh university college of engineering manthani** submitted in partial fulfillment for the award of the degree of bachelor of technology in computer science and engineering(AI&ML).

this is a record of bonofide work carried out by us and the results embodied in this project report have not been reproduced or copied from any source. the results embodied in this project have not been submitted to any other university or institute for the award of any degree or diploma.

| | |
|---|---|
| **CH. AKHIL** | **22VD1A6603** |
| **R. SUMANTH** | **22VD1A6647** |
| **A. SHIVAKUMAR** | **23VD5A6612** |

# JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
# HYDERABAD UNIVERSITY COLLEGE OF ENGINEERING
# MANTHANI
# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AIML)


## CERTIFICATE

This is to certify that the project report entitled "Smart-mart(Online Ordering & Delivery Platform)" being submitted by

| | |
|---|---|
| **CH. AKHIL** | **22VD1A6603** |
| **R. SUMANTH** | **22VD1A6647** |
| **A. SHIVAKUMAR** | **23VD5A6612** |

in the partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **computer science and Engineering(AIML)** to the **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD UNIVERSITY COLLEGE OF ENGINEERING MANTHANI** is a record of bonfide work carried out by them during the academic year 2023-2024.

The results of investigation enclosed in this report have been verified and found satisfactory. The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma.

**SUPERVISER** **HEAD OF THE DEPARTMENT**

# ACKNOWLEDGMENT

We express our sincere gratitude to **Dr. Ch. Sridhar Reddy, Professor of Mechanical Engineering, Principal & Head of the department of Computer Science and Engineering(AIML), JNTUH University College of Engineering Manthani** for encouraging and giving permission to accomplish our project successfully.

We express our profound gratitude and thanks to our project guide **Mr. G. Sridhar, Assistant Professor, Department of Computer Science and Engineering, JNTUH University College of Engineering Manthani** for his constant help, personal supervision, expert guidance and consistent encouragement throughout this project which enabled us to complete our project successfully in time.

We also take this opportunity to thank other faculty members of CSE Department for their kind co-operation.

We wish to convey our thanks to one and all those who have extended their helping hands directly and indirectly in completion of our project.

| | |
|---|---|
| **CH. AKHIL** | **22VD1A6603** |
| **R. SUMANTH** | **22VD1A6647** |
| **A. SHIVAKUMAR** | **23VD5A6612** |

# ABSTRACT

**SMART-MART** is an Online product ordering and delivery platform, With the growth of rapid technology people are busy with their work which affects their household work like, not having minimum time to buy necessary goods to overcome this problem Our team came up with a new idea called Smart-mart.

The customers can buy products at the Online Smart-mart website. The online Smart-mart shopping website to submit selected products and facilities from a Store (or) Local Shops (or) Supermarket that distributes both walk-in-clients and also Cash on deliveries are available.

The online Smart-mart shopping website offers a unique solution that eliminates the problem of delay in delivery, extra money to pay for fast delivery in existing systems, and introducing new delivery techniques in our Smart-mart. By allowing you to choose shops based on your location, Smart-mart ensures faster deliveries at the right address. Since nearby shops fulfill your orders, the distance is minimized, leading to quicker deliveries

The Smart-mart online store system can display all the products they want to Wholesale, from current sector Supermarkets the Smart-mart Online store assists customers in selecting their products once the customer adopts to submit a buying order the Smart-mart online store service center sends the soft copy of a bill list to the particular selected Shop or Supermarket with the user-specified location.

Our online shopping website provides a variety of delivery and time-saving methods, allowing busy people to receive their goods within a short period of time E-commerce has seen tremendous growth in the past decade. our website includes daily usage products, food items, vegetables, fruits, and other products that are used in daily life.

The Online SMART-MART product ordering and delivery platform coded in DJANGO(python framework), CSS, HTML, and JAVASCRIPT are used for fronted and MYSQL, PYTHON for backed Development PYTHON is used as an interface between the web form and Database for storing and fetching.

# TABLE OF CONTENTS

# LIST OF FIGURES

# INTRODUCTION

# 1.INTRODUCTION

## 1.1 INTRODUCTION

In today's world internet is covered as the layer of the world and it can occupy every place in the world. One of business that the internet introduced is an smart-mart website (product Ordering and shopping system). In today's age of fast working system and also increment of stress level people chosen to focus on quick online ordering and especially including speedy delivery of products rather the visual buying of products.

This project is a web-based ordering system for an existing shop. The main aim for developing this project, where customer can place an order on nearby groceries or supermarkets for required products. Customers can easily buy the products from home using this website and this website provides services in Android and various platforms. This software is supported to eliminate and, in some cases, reduce the hardships faced by customers. No technical knowledge is needed for the user to use this systematic website. Thus by this all it proves it is user-friendly interactive website and also provides smart-mart voice assistant to save the customer valuable time.

By this website there is an advantage, the product is directly delivered to customer address by small stores within the 24hours which are chosen by user Convenient supermarkets or stores. This website functionality of the product orders is stored on both as admin and seller side as like soft-copies on web server. This project provides a lot of facilities and features to manage the product in well manner. This project details in advance module that can the back end system very powerful. This website is a method of E-commerce that allows customer to buy a product from store over internet which is chosen by customer availability nearby stores.

People want to go grocery shopping because it's a stress reliever for them. But sometimes, it's a hassle to go shopping and to roam around at the supermarket and look for the product that they will purchase. So, with the use of this technology, it will open the doors to improved customer services through Customer Relationship Management, by offering customers products that is line on their needs, discounts and products recommended based on their purchasing patterns.

## 1.2 PROBLEM STATEMENT

- The current e-commerce system does not utilize the proximity of nearby shops, resulting in longer delivery times and higher costs.
- This inefficiency leads to delays, increased expenses, and wasted time for both customers and delivery personnel.
- Implementing a system that allows customers to purchase products from nearby shops can minimize delivery time and cost, improving overall efficiency and customer satisfaction.

## 1.3 Scope

The Smart Mart e-commerce website aims to help local stores sell their products online easily. It provides a platform where stores can list their items, customers can order them, and everything is managed smoothly. It focuses on making shopping convenient with quick orders, safe payments, personalized features, and useful data for business growth. Smart-mart wants to connect customers with nearby stores effectively, making shopping easier and supporting local businesses.

## 1.4 OBJECTIVES

❖ **Efficient Product Management**

- Enable shop owners to easily add, edit, and delete products.
- Allow for real-time updates of product availability and pricing.
- Implement features to enable or disable products based on stock status.

❖ **User-Friendly Shopping Experience**

- Provide a seamless and intuitive shopping interface for customers.
- Implement an easy-to-use search functionality to find products and shops.
- Offer detailed product descriptions, images, and reviews.

❖ **Enhanced Customer Convenience**

- Allow customers to buy products from nearby shops to minimize delivery time and cost.
- Provide multiple payment options, including credit/debit cards, UPI, and QR code payments.
- Send OTP's for verification during registration and order deliveries for security.

❖ **Effective Delivery Management**

- Build a system for delivery users to manage orders, including login, registration, and tracking.
- Integrate maps for route optimization between shop and customer.
- Allow delivery boys to update order status and handle OTP verification for deliveries.

- **Robust Payment Processing**
  - Integrate reliable payment gateways like Razorpay for secure transactions.
  - Ensure payment response handling with signature verification for security.
  - Offer a clear and informative payment success page.
- **Smart Assistance**
  - Implement a Smart Assistant feature to help users with shopping queries and navigation.
  - Include voice search capabilities to enhance user experience.
  - Display a modal with the Smart Assistant when activated.
- **Customer Feedback and Support**
  - Provide a feedback form with an appealing design, including animations and transitions.
  - Ensure customer queries and feedback are addressed promptly.

## 1.5 MOTIVE

- The motive behind developing the Smart-mart e-commerce web application is to enhance-local stores' growth by providing them with a robust online platform.
- This platform aims to enable customers to access a wide range of products at competitive prices, delivered quickly and efficiently.
- Additionally, Smart-mart seeks to add value through additional features that simplify shopping experiences and foster strong connections between local businesses and their communities.
- This initiative seeks to not only simplify daily shopping routines but also promote economic growth and sustainability within local communities.

# LITERATURE SURVEY

# 2. LITERATURE SURVEY

The rapid adoption of online shopping has reshaped consumer behavior, emphasizing convenience and accessibility. Platforms like Amazon and Flip-kart have set high standards with user-friendly interfaces and seamless navigation.

❖ **HTML, CSS, and JavaScript for Fronted Development**

The surge in online shopping underscores the importance of a user-friendly interface and seamless navigation. HTML (Hypertext Markup Language**)** serves as the foundation of Smart Mart's fronted, structuring web content with elements like text, images, and links. CSS (Cascading Style Sheets) enhances the presentation of HTML elements, ensuring consistent styling across different devices and browsers. This combination creates a visually appealing and responsive design, crucial for delivering a compelling user experience. JavaScript adds interactivity to Smart Mart's fronted, enabling features such as dynamic content updates, client-side form validation, and interactive elements that enhance usability and engagement.

❖ **Django-Python Framework for Backed Development**

Behind the scenes, Django, a powerful Python web framework, drives Smart-mart's backed operations. Python, known for its readability and versatility, supports rapid development and clean, pragmatic design. Django's built-in features include authentication, URL routing, and an ORM (Object-Relational Mapping) system, which simplifies database interactions. This framework facilitates the efficient management of business logic, ensuring scalability and performance across Smart Mart's e-commerce platform.

❖ **MySQL Database Management System**

MySQL, a robust relational database management system, underpins Smart-mart's data management strategy. It offers secure and efficient storage for user profiles, product catalogs, transaction records, and more. MySQL's relational structure supports complex queries and transactions, essential for handling the diverse needs of Smart-mart's store network. This integration ensures data integrity, scalability, and reliable performance, crucial for supporting Smart-mart's growth and operational demands.

❖ **User Experience (UX) Design Principles**

User experience (UX) is a cornerstone of Smart-mart's development strategy, focusing on intuitive navigation, responsive design, and streamlined checkout processes. By adhering to UX design principles, Smart-mart aims to optimize customer satisfaction, increase conversion rates, and foster long-term customer loyalty. The intuitive interface and seamless user journey enhance usability, making shopping on Smart-mart enjoyable and efficient for users.

❖ **Integration of Systems and Web Services**

Integration is key to Smart-mart's operational efficiency, connecting Django-powered modules such as Online Ordering System Login (OSL), Billing System, and Customer Relationship Management (CRM) through web services. This cohesive integration streamlines store management operations, facilitating efficient order processing, inventory management, and customer support. By ensuring seamless connectivity between systems, Smart-mart delivers a unified user experience and operational synergy across its e-commerce platform.

In conclusion, the literature survey underscores how Smart-mart integrates HTML, CSS, JavaScript, Django-Python, and MySQL to develop a robust and user-eccentric e-commerce platform. By leveraging these technologies effectively, Smart-mart aims to deliver a seamless online shopping experience that meets modern consumer expectations while maintaining scalability and operational efficiency across its network of stores.

# SYSTEM ANALYSIS

# 3.SYSTEM ANALYSIS

## 3.1 EXISTING SYSTEM

E-commerce systems are existing online platforms where people can shop for goods and services. These platforms include websites and apps that allow users to browse products, make secure purchases, and track deliveries. They manage payments, handle inventory, and provide customer support. Major platforms like Amazon and eBay use advanced tools to suggest products and run promotions, making shopping easier and more personalized. E-commerce systems play a crucial role in modern retail, offering convenience and accessibility while connecting buyers and sellers worldwide.

## 3.2 LIMITATIONS OF EXISTING SYSTEM

- Slow delivery times impacting customer satisfaction.
- High shipping costs deterring purchases.
- Limited product visibility due to poor search functionality.
- Difficulty in managing returns and refunds efficiently.
- Dependency on third-party logistics causing delays.
- Lack of personalized shopping experiences affecting customer retention.
- Regulatory compliance challenges across different regions.
- High cost for fast delivery.

## 3.3 PROPOSED SYSTEM

The Smart Mart project aims to revolutionize local shopping by integrating an efficient e-commerce platform. It connects nearby stores with customers, ensuring quick and cost-effective delivery. Key features include real-time product availability, seamless payment options, and optimized delivery routes using smart mapping technology. The project not only enhances local stores' visibility but also prioritizes customer convenience through a user-friendly interface and personalized shopping experiences.

## 3.3.1 ARCHITECTURE OVERVIEW

❖ **Users and Their Roles**

The system features three primary user roles: Customer, Shop Owner, and Delivery User. The Customer is responsible for placing orders within the system. The Shop Owner manages the orders, products, and categories within the shop. The Delivery User is tasked with picking up and delivering orders. These user roles form the basis of the system's operations, ensuring that each action within the order life cycle is handled by the appropriate user type.

❖ **User Actions and Order Management**

Each user role is associated with specific actions. The Customer performs the "place" action to create an order. The Shop Owner is linked to orders through the "has" action, indicating their responsibility for managing the orders. The Delivery-user performs the "pick" action, signifying their role in handling the orders for delivery. This clear definition of actions ensures a streamlined process for order management and fulfillment within the Smart Mart system.

❖ **Core Component: Order**

At the heart of the system is the Order component, which acts as the central entity connecting users to services. An order can be placed by a customer, managed by a shop owner, and picked up by a delivery user. This centralization of order management ensures that all user actions and services revolve around a single entity, facilitating efficient tracking and processing of orders from creation to delivery.

❖ **Supporting Services**

The Smart-mart system includes essential services such as Payment and Order_Cancel. The Payment service is directly linked to the order component, handling the financial transactions associated with each order. The Order_Cancel service allows for the cancellation of orders, providing flexibility and control over order management. These services are crucial for maintaining the integrity and functionality of the order process, ensuring that all necessary operations can be performed seamlessly.

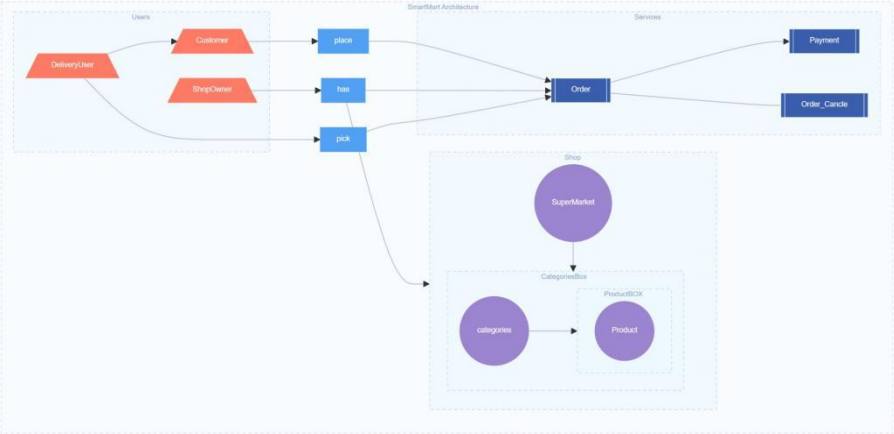


Fig 3.1 System Architecture

### 3.3.2 Advantages

- Much faster transactions available 24/7.
- Robust and provides high security.
- Cash on collect method.
- Deliveries the products within 24hours
- User can order the items from selected supermarkets or store(choice of globality) within the geographical area.
- Products and services are easy to find.
- Speeding up the national economic development
- Enhancing Technology development in villages
- Easier time managing a business.
- Does not require any physical space.
- No geographical limitations mean a bigger customer reach.
- Higher quality of services and lower operational costs.

## 3.4 FEASIBILITY STUDY

A feasibility study for Smart Mart, focusing on enhancing local stores' growth through an e-commerce platform, would typically include the following aspects:

- **Market Feasibility:**
  - **Market Analysis:** Evaluate the demand for local products online and the potential customer base.
  - **Competitive Analysis:** Assess existing e-commerce platforms and their offerings in the local market segment.
  - **Target Audience:** Define the demographics and preferences of potential customers who prefer local products.
- **Technical Feasibility:**
  - **Platform Selection:** Evaluate the suitability of technologies (e.g., Django for backed, React for fronted) for building the e-commerce platform.
  - **Infrastructure Requirements:** Assess the necessary hardware and software infrastructure for hosting and maintaining the platform.
  - **Security Considerations:** Ensure robust measures for user data protection, payment security, and secure transactions.
- **Environmental and Social Feasibility:**
  - **Impact Assessment:** Evaluate the environmental and social impact of promoting local products and supporting local businesses.
  - **Sustainability:** Incorporate sustainable practices in operations, such as eco-friendly packaging and efficient delivery routes.

# SYSTEM REQUIREMENTS SPECIFICATION

# 4. SYSTEM REQUIREMENT SPECIFICATION

## 4.1 INTRODUCTION

Software Requirements Specification plays an important role in creating quality software solutions. Specification is basically a representation process. Requirements are represented in a manner that ultimately leads to successful software implementation. Requirements may be specified in a variety of ways. However, there are some guidelines worth following:

- Representation format and content should be relevant to the problem.
- Information contained within the specification should be nested
- Diagrams and other notation forms should be restricted in number and consistent in use.

## 4.2 FUNCTIONAL REQUIREMENTS

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks. So, it's important to make them clear both for the development team and the stakeholders. Generally, functional requirements describe system behaviour under specific conditions

## 4.3 NON-FUNCTIONAL REQUIREMENTS:

- **USABILITY**

  **Intuitive Interface:** User-friendly design for easy navigation.

  **Accessibility:** Support accessibility standards (e.g., WCAG) for users with disabilities.

  **Multi-language Support:** Provide language options for diverse user groups.

- **PERFORMANCE**

  **Response Time:** System should respond to user actions within 2 seconds.

  **Scalability:** Support up to 10,000 concurrent users during peak times.

  **Reliability:** Ensure 99.9% uptime, with minimal downtime for maintenance.

- **SECURITY**

**Data Protection:** Encrypt sensitive user data (e.g., passwords, payment information).

**Access Control:** Implement role-based access control (RBAC) for different user roles.

**Secure Payments:** Use HTTPS and PCI DSS compliant payment gateways

## 4.4 HARDWARE REQUIREMENTS

- Processor: I5 processor 11th Generation.
- Random Access Memory (RAM): 4GB or more.
- Solid State Drive (SSD): 512GB

## 4.5 SOFTWARE REQUIREMENTS

- Operating System: Windows 11.
- Programming languages:
  - ✓ Fronted:- HTML,CSS,JAVASCRIPT
  - ✓ Backed:- PYTHON - DJANGO(framework).
  - ✓ Database:- MYSQL.
- Web Browser: Latest version of Chrome.
- Platform: visual studio code

### 4.5.1 VISUAL STUDIO CODE

Visual Studio Code (VS Code) is a versatile, free source-code editor by Microsoft. It supports multiple languages, offering smart code completion, debugging, and a vast extension library for tailored experiences. Its intuitive interface includes a sidebar for easy navigation and an integrated terminal. VS Code excels in debugging, featuring breakpoints and variable inspection.

Using Visual Studio Code (VS Code) for developing Smart-mart, an e-commerce platform, begins with setting up the development environment. Start by downloading and installing VS Code, along with Python and Django. Create a virtual environment to manage dependencies and keep your project organized.

VS Code's extensive extension library enhances the development process for Smart Mart. Install essential extensions such as the Python extension for code completion and debugging, Django extension for improved framework support, and Git Lens for powerful Git integration. Utilize the integrated terminal to run Django commands, manage virtual environments, and execute Git commands without leaving the editor. Leverage VS Code’s debugging tools to set breakpoints, inspect variables, and step through your code, which simplifies troubleshooting and enhances productivity. With these features, VS Code provides a comprehensive and efficient environment for building and maintaining the Smart-mart platform.

# SYSTEM DESIGN

# 5. SYSTEM DESIGN

## 5.1 INTRODUCTION

System design in the context of Smart-mart involves defining its essential elements such as architecture, modules, components, interfaces, and data flow. This phase is pivotal in ensuring that Smart-mart meets the specific needs and requirements of its users—shop owners, delivery personnel, and customers through a well-structured and functional platform.

The system design phase of Smart-mart is both creative and challenging, as it lays the foundation for the final platform. It encompasses detailing technical specifications that guide the implementation of features such as product management, order processing, delivery logistics, and customer interaction.

The significance of system design for Smart-mart lies in its ability to ensure quality. A well-designed system provides a blueprint that can be assessed and refined to meet high standards of performance, reliability, and user satisfaction. It serves as the bridge between customer requirements and the actual realization of a cohesive and effective e-commerce solution. Without a robust design, Smart-mart risks instability, difficulty in testing, and potential challenges in maintaining and scaling the platform effectively.

In summary, system design is an indispensable phase in the development of Smart-mart, ensuring that it evolves into a reliable and saleable e-commerce platform that enhances local businesses and provides a seamless experience for its users.

## 5.2 UML DIAGRAMS

UML is a way of visualizing a software program using a collection of diagrams. The notation has evolved from the work of the Rational Software Corporation to be used for object-oriented design, but it has since been extended to cover a wider variety of software engineering projects. The Object Management Group as the standard for modeling software development accepts UML.

UML stands for Unified Modelling Language. UML 2.0 helped extend the original UML specification to cover a wider portion of software development efforts including agile practices.

- Improved integration between structural models like class diagrams and behaviour models like activity diagrams.
- Added to the ability to define a hierarchy and decompose a software system into components and sub components.
- The original UML specifies nine diagrams; UML 2. x brings that number up to13.The four new diagrams are called: Communication diagram, a composite structure diagram, an interaction overview diagram, and a timing diagram. It also renamed state chart diagrams to state machine diagrams, also known as state diagrams.

## 5.3 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. A use case diagram can identify the different types of users of a system and the different use cases and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

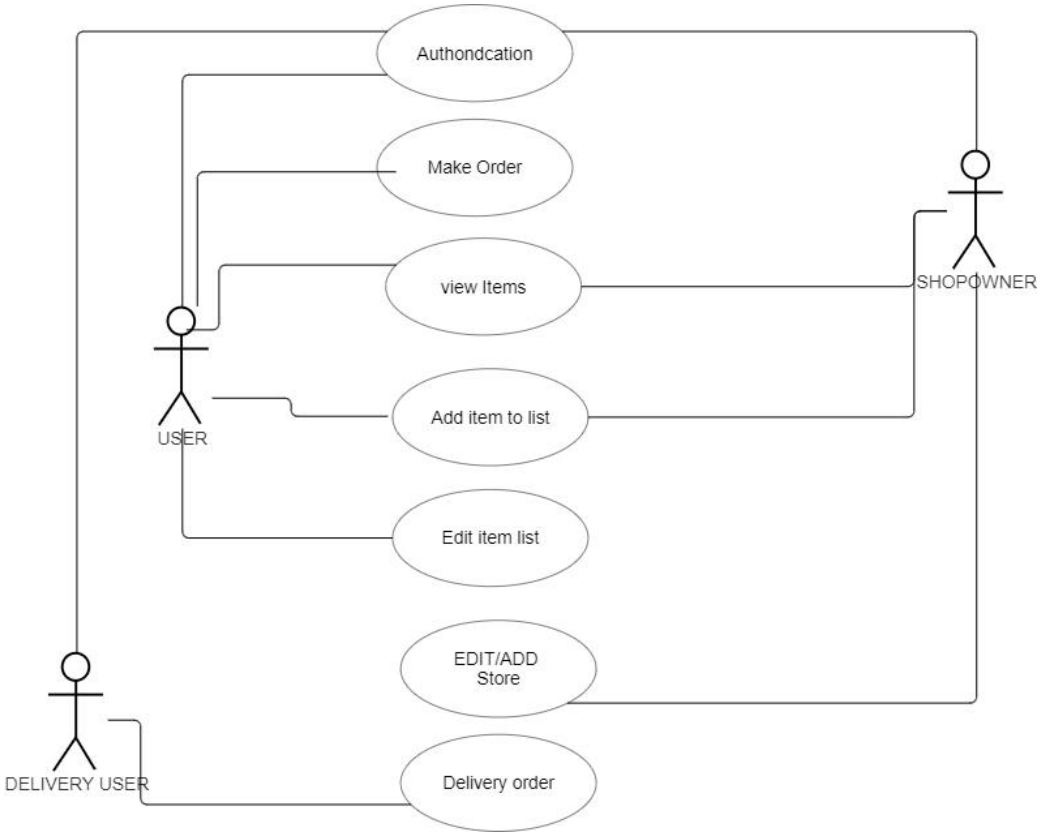


Fig 5.2.1 Use Case Diagram

## 5.4 ACTIVITY DIAGRAM

Activity diagram is sometimes considered as the flowchart. Although the diagrams look like a flowchart, they are not. It shows different flows such as parallel, branched, concurrent, and single. Activity is a particular operation of the system. The below image represents the workflow or activity diagram of the Cursor and keyboard activity diagram.

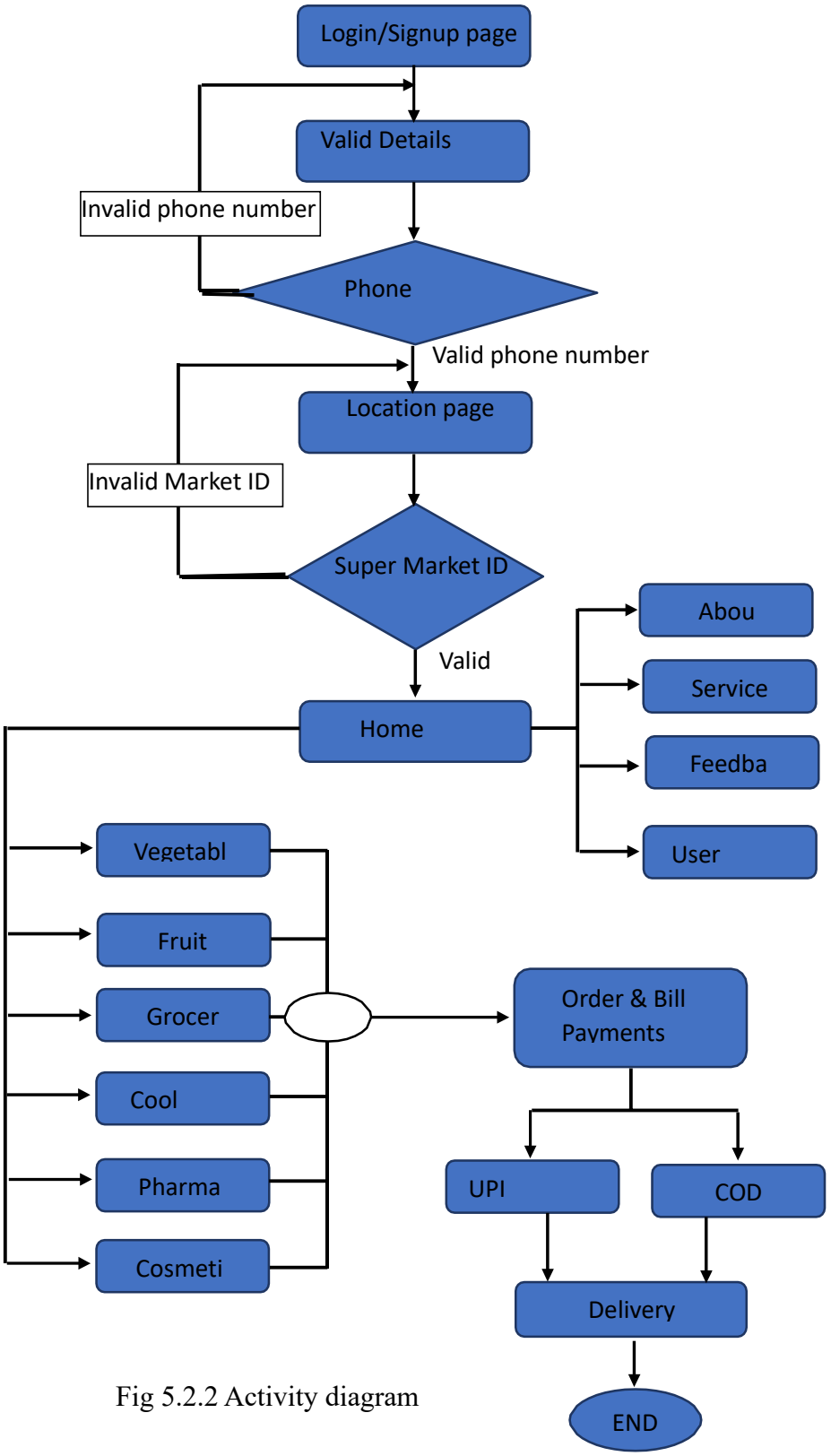


Fig 5.2.2 Activity diagram

## 5.5 CLASS DIAGRAM

A class diagram is a fundamental component of Unified Modeling Language (UML) used in software engineering to visualize the structure of a system in terms of classes, their attributes, and the relationships between classes in it. Associations between classes are represented by lines, showcasing relationships such as composition, aggregation, or simple associations. Inheritance, denoted by solid lines with hollow arrowheads, illustrates the "is-a" relationship between a super-class and its sub-classes. Interfaces, circles adjacent to classes, specify a contract for implementing classes.

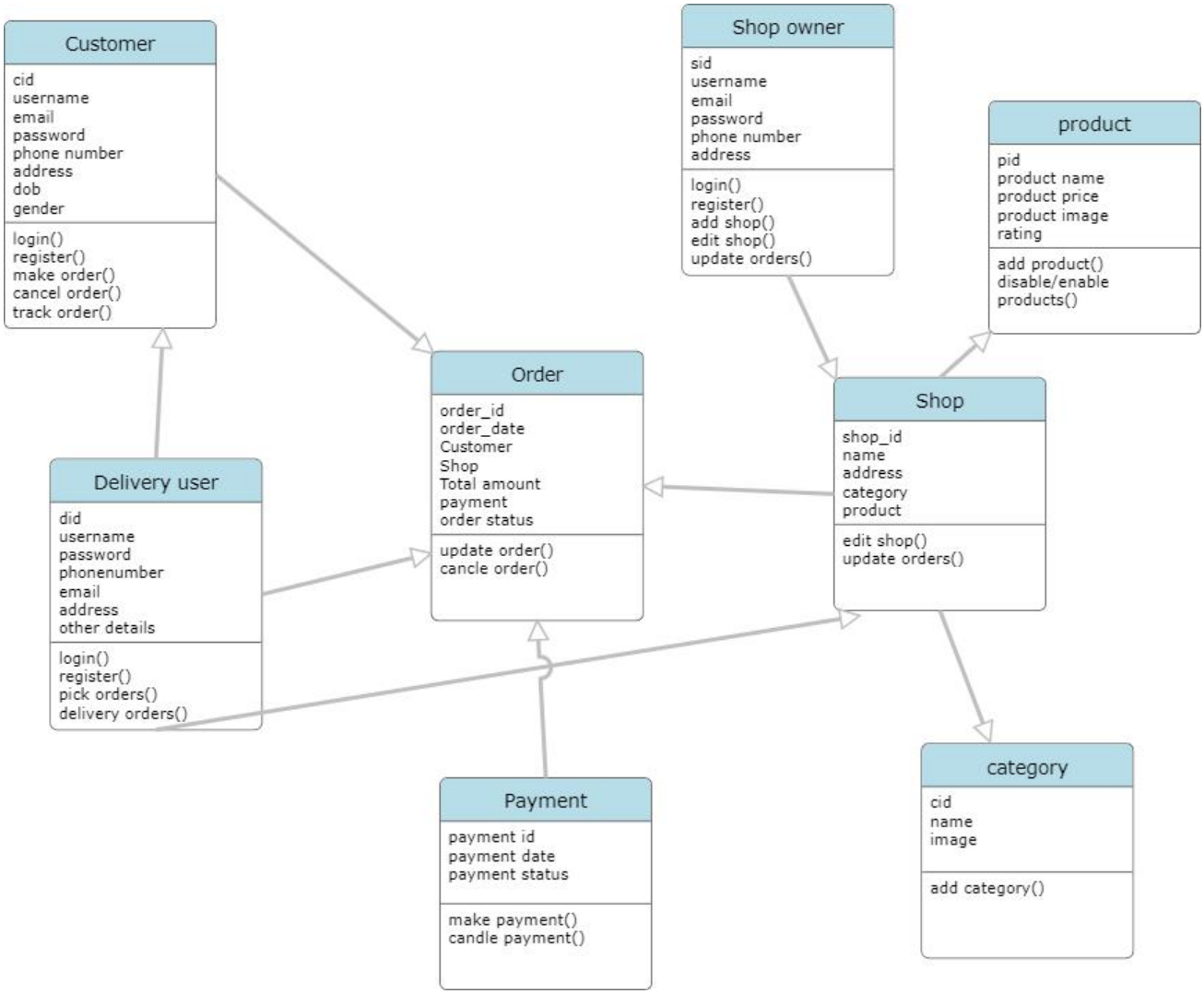


Fig 5.2.3 Class Diagram.

## 5.6 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a visual representation that helps us understand how information moves within a system. It shows how data enters and exits the system, what processes or actions change the data, and where the data is stored. A well-designed DFD can provide a clear pictureof the system's requirements. It can be created manually, using pen and paper, or using software tools.The diagram helps define the scope and boundaries of the system. It can also be used as a communication tool between a systems analyst and anyone involved in the system. Additionally, it serves as a starting point for redesigning a system by identifying areas that need improvement.

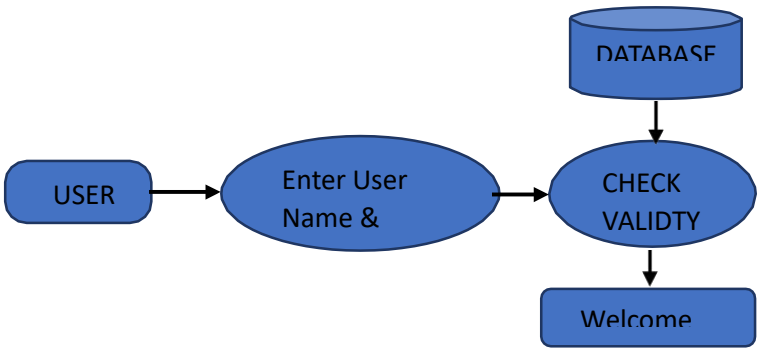


Fig 5.2.4 Login DFD

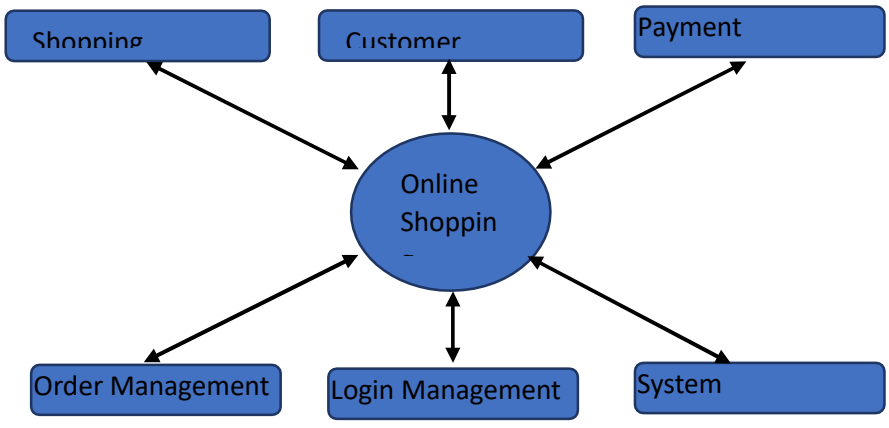


Fig 5.2.5 Requirements Flow Diagram.

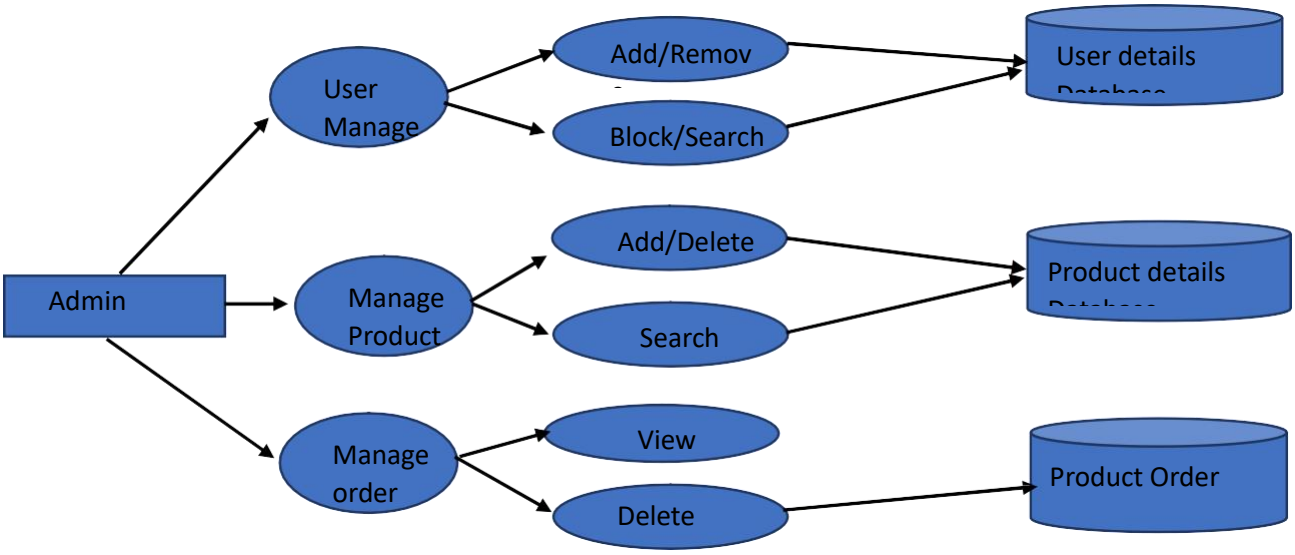


Fig 5.2.5 ADMIN Data Flow Diagram.

## 5.7 SYSTEM ARCHITECTURE

An architecture model is a partial abstraction of a system. It is an approximation, and it captures the different properties of the system. It is a scaled-down version and is built with all the essential details of the system. Architecture modelling involves identifying the characteristics of the system and expressing it as models so that the system can be understood. Architecture models allow visualization of information about the system represented by the model

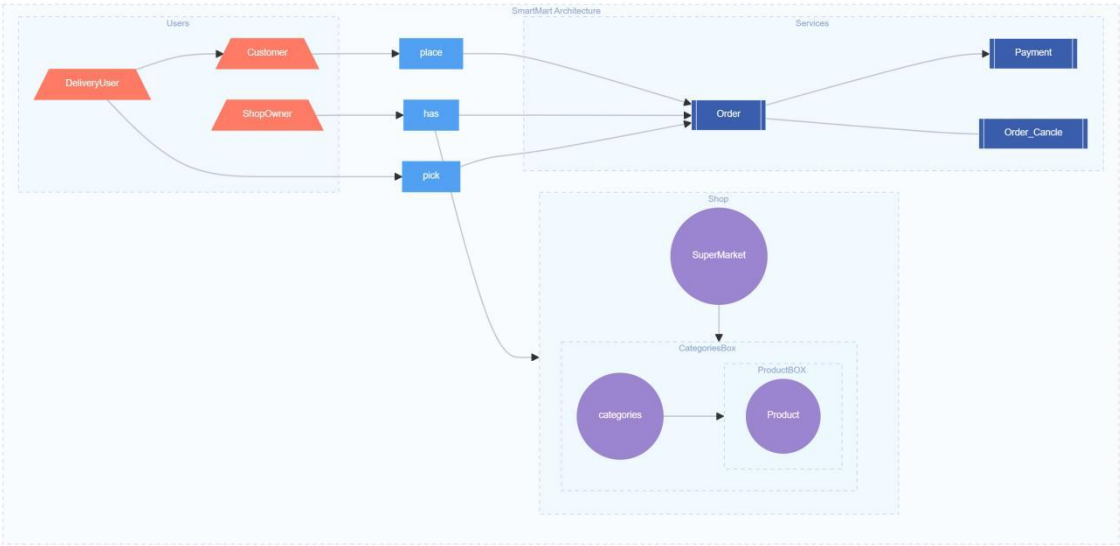


Fig 5.3 System Architecture

# IMPLEMENTATION

# 6.IMPLEMENTATION

## 6.1 USER IMPLEMENTATION

The user interface of the application has been designed using Microsoft Visual Studio.The user can see the list of products that are available. The user can search for products by entering the search term into the search text-box provided on the top. This text-box is watermarked with the words “Search” to let the user know that this is the place to enter the search terms. The user can filter the products by using the drop-down lists.

The user can move the cursor on to the small images to view the same image in the enlarged position. The user can click on the enlarged picture to see a still bigger image. Similarly, a user can click on the tab panel customer reviews, specifications, manufacturer Info etc. to see the respective information.

The potential users of the system are administrators and customers of OOS. The customer will enter the require information in the order form. Also, the administrator must fill the form with required information. However, the software and build order. The administrator will have familiarity with the way the order should be filled. The customer programs to an expert.

**USER FUNCTION**

- Registered User: (Customer. Administrator)
- Login system (Username and Password)

❖ **CUSTOMER**

- Fill the form (with the required Customer information
- Display old orders and select one of the them to display it items
- Start new orders and select required product and quantity
- Fill payment detail and sent order.
- Display product details and add new products
- Display category list and can add/modify them
- display payment type

- Register new administrator employees (create user IDs for the admin and employees in-order to login)
- Change current user password and after login
- Display customer orders

## 6.2 SHIPOWNER IMPLEMENTATION

Shop Owners using Smart-mart have access to comprehensive documentation that guides them through setting up and managing their online stores efficiently. The documentation covers essential operations such as:

- Setting up a Shop Owner account and profile.
- Adding products to their store catalog with detailed descriptions and images.
- Organizing products into categories for better customer navigation.
- Viewing and managing incoming orders, including order confirmation and status updates.
- Creating and managing promotional campaigns, discounts, and coupon codes.
- Customizing store settings such as name, contact details, and operating hours to reflect their brand identity.

## 6.3 DELIVERY USER IMPLEMENTATION

Delivery Users on Smart-mart are provided with clear documentation to efficiently manage their delivery operations. Key operations include:

- **Account Setup:** Guide on creating a Delivery User account and setting up profile information.
- **Order Assignment:** Instructions for receiving and assigning orders for delivery.
- **Navigation Assistance:** Integration with maps for optimal route planning between shops and customers.
- **Order Verification:** Process for verifying orders upon pickup and ensuring accuracy.
- **Customer Interaction:** Guidelines on interacting professionally with customers during delivery.

- **Status Updates:** Updating order status within the Smart-mart platform post-delivery.

## 6.4 IMPLEMENTATION OF SMART MART ASSISTANT

In addition to the best answer of the query, it also returns with an approximate confidence from the model. Reusable generic module which captures voice input and convert it into text and convert the text back to voice once the result received from the server. It is implemented using HTML and JavaScript library.

- **ADVANTAGES**
  - ✓ Smart-mart Voice assistant web app provides hands-free experience to answer the quickqueries
  - ✓ Saves the precious time
  - ✓ Fast response
  - ✓ Easy to use

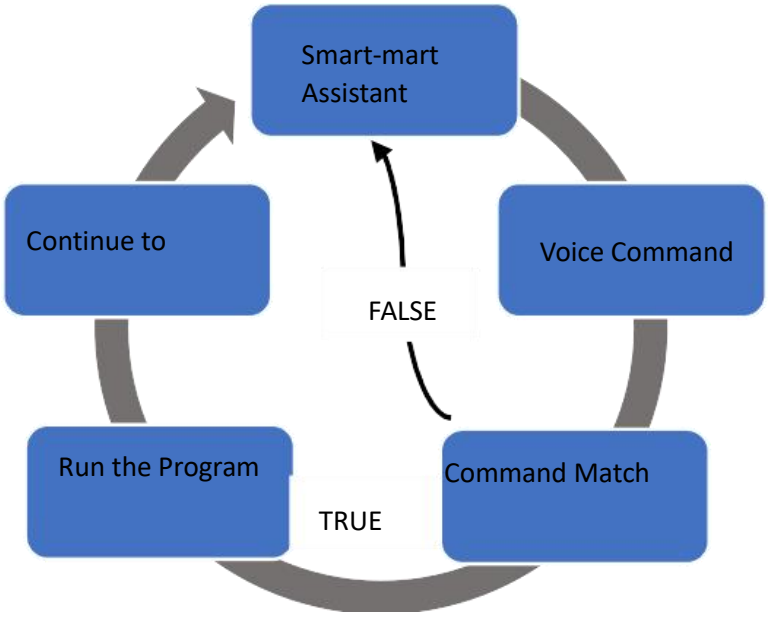


Fig 6.4.1 Assistant Diagram

## 6.5. SAMPLE CODE

### ✧ USER REGISTER CODE

```
from django.contrib.auth.models import User

from .models import mainuser

def user_form_view(request):

    if request.method == 'POST':

        user_fname = request.POST['fname']

        print(user_fname)

        user_lname = request.POST['lname']

        user_mail = request.POST['email']

        user_password = request.POST['pass']

        user_dob = request.POST['dob'];

        user_phonenumber = request.POST['num']

        en_password = encode_password(user_password)

        en_password=encode_password(user_password)

        # Check if a user with the same email already exists

        if mainuser.objects.filter(user_mail=user_mail).exists():

            error_message = "A user with this email already exists. Please use a different email."

            return render(request, 'signup.html', {'error_message': error_message})

        # Create a new User instance

        user = User.objects.create_user(username=user_lname, email=user_mail, password=user_password)

        user.first_name = user_fname

        user.last_name = user_lname

        user.is_active = False  # User is inactive until email confirmation

        user.save()

        # Create a new MainUser instance

        new_user = mainuser(


            user_fname=user_fname,

            user_lname=user_lname,

            user_mail=user_mail,

            user_password=en_password,

            user_dob=user_dob,

            user_phonenumber=user_phonenumber

        )

        new_user.save()

        # Send confirmation email

        current_site = get_current_site(request)

        mail_subject = 'Activate your account.'
```

```
        message = render_to_string('acc_active_email.html', {
            'user': user,
            'domain': current_site.domain,
            'uid': urlsafe_base64_encode(force_bytes(user.pk)),
            'token': default_token_generator.make_token(user),
        })
        html_message = f"""
            {message}
            """
        sendemail = EmailMessage(mail_subject, html_message, email_from, [user_mail])
        sendemail.content_subtype = 'html'  # Set the content type to HTML
        sendemail.send()
        return render(request,'requestdone.html',{'user':new_user})
    else:
        return render(request, 'signup.html')


def activate(request, uidb64, token):
    try:
        uid = force_str(urlsafe_base64_decode(uidb64))
        user = User.objects.get(pk=uid)
    except (TypeError, ValueError, OverflowError, User.DoesNotExist):
        user = None
    if user is not None and default_token_generator.check_token(user, token):
        user.is_active = True
        user.save()
        return render(request, 'gotologin.html')
    else:
        return HttpResponse('Activation link is invalid!')
```

## ✧ USER LOGIN CODE

```
def login_operation(request):
    error_message1 = None
    error_message2 = None

    if request.method == 'POST':
        user_email = request.POST.get('mail')
        user_pass = request.POST.get('password')
```

```
        try:
            user = mainuser.objects.get(user_mail=user_email)
            stored_password= base64.b64decode(user.user_password).decode('utf-8')
            print(stored_password)
            if user_pass == stored_password:
                request.session['user_id'] = user.user_id
                success_message = "Successfully logged in."
                return render(request, 'home.html',{'success_message': success_message,
          'username':user.user_fname+user.user_lname})
            else:
                 error_message1 = "Invalid password."
                 return render(request, 'login.html', {'error_message1': error_message1})
        except mainuser.DoesNotExist:
            error_message2 = "User with this email does not exist."
            return render(request, 'login.html', {'error_message2': error_message2})
    else:
        # GET request, render the login page
        return render(request, 'login.html')
```

## ✧ CREATE ORDER CODE

```
def create_order(request):
    if request.method == 'POST':
        selected_products = request.session.get('selected_products', [])
        shop_id = request.session.get('shop_id')
        user_id = request.session.get('user_id')
        shop = get_object_or_404(SuperMarket, s_id=shop_id)
        user = get_object_or_404(mainuser, user_id=user_id)
        shop_owner = shop.owner
        # Create the order
        total_price = sum(product['product_price'] * product['quantity'] for product in selected_products)
        order = Order.objects.create(
            user=user,
            shop_owner=shop_owner,
            supermarket=shop,
            total_price=total_price
        )
        shopmail=shop.s_mail
        usermail=user.user_mail
        request.session['amount'] = total_price
```

```
        request.session['order_id']=order.order_id
        request.session['user_mail']=user.user_mail
        for item in selected_products:
            product = get_object_or_404(Product, product_id=item['product_id'])
            OrderItem.objects.create(
                order=order,
                product=product,
                quantity=item['quantity'],
                price=product.product_price * item['quantity']
            )
        request.session['selected_products'] = []
        request.session['shopmail'] = shopmail
        request.session['usermail'] = usermail
    return redirect('display_selected_products')
```

## ✧ TRACK ORDER CODE

```
from django.shortcuts import render, get_object_or_404, redirect
from .models import Order, mainuser, OrderUpdate
def track_order(request):
    if request.method == 'POST':
        order_id = request.POST.get('order_id')
        user_id = request.session.get('user_id')
        order = get_object_or_404(Order, pk=order_id,user=user_id)
        updates = OrderUpdate.objects.filter(order=order).order_by('update_timestamp')

        return render(request, 'track_order.html', {'order': order, 'updates': updates})
    else:
        return redirect('user_orders')
```

## MAP CODE

```
def maps(request):
    supermarkets = SuperMarket.objects.all()

    shops = []
    for supermarket in supermarkets:
        shop_data = {
            "id": supermarket.s_id,
            "name": supermarket.s_name,
            "mail": supermarket.s_mail,
            "lat": supermarket.latitude,
            "lng": supermarket.longitude,
            "type": supermarket.s_type,
            "address": supermarket.s_address,
            "opentime": time_to_string(supermarket.s_opentime),
            "closetime": time_to_string(supermarket.s_closetime),
        }
        shops.append(shop_data)
        message='done'
    context = {
        'shops': json.dumps(shops),
        'message':message# Serialize shops data to JSON
    }

    return render(request, 'maps.html', context)
```

## ✧ MAP UI CODE

```
<!DOCTYPE html>
<html>
<head>
    <title>Nearby Shops</title>
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.7.1/dist/leaflet.css" />
</head>
<body>
    <a href="/home" class="home-link">Home</a>
    <div class="container">
        <h1>Maps Page - Select Shop</h1>
```

```
    <div class="shop-search-section">
        <form id="searchForm">
            <label for="searchTerm">Search Shops:</label>
            <input type="text" id="searchTerm" name="searchTerm" placeholder="Enter shop name or type...">
            <button type="button" onclick="searchShops()" class="btn">Search</button>
        </form>
    </div>
    <div id="map"></div>
    <div class="shop-details" id="shopDetails"></div>
    <div class="shop-id-section">
        <form action="{% url 'display_categories' %}" method="POST">
            {% csrf_token %}
            <label for="shopid">Enter Shop_ID:</label>
            <input type="text" name="shopid" id="shopid">
            <button type="submit" class="btn">Submit</button>
            {% if error_message1 %}
                <p class="error-message">{{ error_message1 }}</p>
            {% endif %}
        </form>
    </div>
</div>
<div class="smart-assistant-form">
    <form action="{% url 'smart' %}" method="post">
        {% csrf_token %}
        <div class="form-group">
            <button type="submit" id="assistant-button">Smart Assistant <i class="fa fa-microphone"></i></button>
        </div>
    </form>
</div>


{% if message %}
<script>
    document.addEventListener('DOMContentLoaded', function() {
        function speak(message) {
            const speech = new SpeechSynthesisUtterance(message);
            speech.rate = 1;
            speech.pitch = 1;
            window.speechSynthesis.speak(speech);
        }
        speak("select nearby shop or Search your shop");
```

```
    });
</script>
{% endif %}
<div class="footer">
    <p>&copy; 2024 Smart-mart. All rights reserved.</p>
</div>
<script src="https://unpkg.com/leaflet@1.7.1/dist/leaflet.js"></script>
<script>
    var map;
    var markers = [];
    //var userLocation;
    function initMap() {
        var defaultLocation = [18.6439, 79.5475]; // Coordinates for JNTUH Manthani@Centenary Colony-CANTEEN
        map = L.map('map').setView(defaultLocation, 15);
        L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
            maxZoom: 12,
        }).addTo(map);
        var defaultMarker = L.marker(defaultLocation).addTo(map)
            .bindPopup('Current location')
            .openPopup();
        var circle = L.circle(defaultLocation, {
            color: 'red',
            fillColor: '#f03',
            fillOpacity: 0.5,
            radius: 10000 // 10 km radius
        }).addTo(map);
        var shops = JSON.parse('{{ shops|escapejs }}');
        for (var i = 0; i < shops.length; i++) {
            addMarker(shops[i]);
        }

      //  getUserLocation();
    }
    function addMarker(shop) {
        var marker = L.marker([shop.lat, shop.lng]).addTo(map)
            .bindPopup('<strong>' + shop.name + '</strong><br>Shop ID: ' + shop.id);
        marker.shop = shop;
        marker.on('click', function() {
            showShopDetails(shop);
        });
```

```
            markers.push(marker);
        }
        function showShopDetails(shop) {
            var shopDetailsContainer = document.getElementById('shopDetails');
            shopDetailsContainer.innerHTML = '';
            var shopInfoDiv = document.createElement('div');
            shopInfoDiv.classList.add('shop-info');
            var shopName = document.createElement('h3');
            shopName.textContent = shop.name;
            shopInfoDiv.appendChild(shopName);
            var shopId = document.createElement('p');
            shopId.textContent = 'Shop ID: ' + shop.id;
            shopInfoDiv.appendChild(shopId);
            var shopType = document.createElement('p');
            shopType.textContent = 'Type: ' + shop.type;
            shopInfoDiv.appendChild(shopType);
            var shopAddress = document.createElement('p');
            shopAddress.textContent = 'Address: ' + shop.address;
            shopInfoDiv.appendChild(shopAddress);
            var shopOpenTime = document.createElement('p');
            shopOpenTime.textContent = 'Open Time: ' + shop.opentime;
            shopInfoDiv.appendChild(shopOpenTime);
            var shopCloseTime = document.createElement('p');
            shopCloseTime.textContent = 'Close Time: ' + shop.closetime;
            shopInfoDiv.appendChild(shopCloseTime);
            shopDetailsContainer.appendChild(shopInfoDiv);
        }
        function searchShops() {
            var searchTerm = document.getElementById('searchTerm').value.toLowerCase();
            var results = [];
            for (var i = 0; i < markers.length; i++) {
                var marker = markers[i];
                var shop = marker.shop;
                if (shop.name.toLowerCase().includes(searchTerm) || shop.type.toLowerCase().includes(searchTerm)) {
                    marker.addTo(map);
                    results.push(marker);
                    showShopDetails(results[0].shop);
                } else {
                    map.removeLayer(marker);
                }
```

```
            }
            if (results.length > 0) {
                map.setView(defaultLocation, 15);
                showShopDetails(results[0].shop);

            } else {
                alert('Shop not found.');
            }
        }
        function getUserLocation() {
            if (navigator.geolocation) {
                navigator.geolocation.getCurrentPosition(function(position) {
                    userLocation = [position.coords.latitude, position.coords.longitude];
                    map.setView(userLocation, 15);
                    L.marker(userLocation).addTo(map).bindPopup('You are here').openPopup();
                }, function() {
                    alert('Geolocation failed. Using default location.');
                });
            } else {
                alert('Geolocation is not supported by this browser. Using default location.');
            }
        }
        document.addEventListener('DOMContentLoaded', function() {
            initMap();
        });
    </script>
</body>
</html>
```

## ✧ VOICE ASSISTANT CODE

```
const btn = document.querySelector('.talk');
const content = document.querySelector('.content');

function speak(sentence) {
    const text_speak = new SpeechSynthesisUtterance(sentence);
    text_speak.rate = 1;
    text_speak.pitch = 1;
    window.speechSynthesis.speak(text_speak);
}
```

```
function wishMe() {
    var day = new Date();
    var hr = day.getHours();
    if (hr >= 0 && hr < 12) {
        speak("Good Morning, Boss");
    } else if (hr == 12) {
        speak("Good noon,  Boss");
    } else if (hr > 12 && hr <= 17) {
        speak("Good Afternoon, Boss");
    } else {
        speak("Good Evening,  Boss");
    }
}
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
const recognition = new SpeechRecognition();
let assistantActive = true;
recognition.onresult = (event) => {
    const current = event.resultIndex;
    let transcript = event.results[current][0].transcript.toLowerCase();

    // Remove trailing punctuation
    transcript = transcript.replace(/[.,\/#!$%\^&\*;:{}=\-_`~()]/g, "").trim();

    content.textContent = transcript;
    speakThis(transcript);
};
recognition.onend = () => {
    if (assistantActive) {
        recognition.start();
    }
};
function speakThis(message) {
    const speech = new SpeechSynthesisUtterance();
    speech.text = "I did not understand what you said, please try again";
    if (message.includes('hey') || message.includes('hello')) {
        speech.text = "Hello Boss, this is Smart-mart";
    } else if (message.includes('how are you')) {
        speech.text = "I am fine, " + username + ", tell me how can I help you";
    } else if (message.includes('name') || message.includes('what is your name')) {
        speech.text = "My name is Smart-mart voice Assistant";
```

```
    } else if (message.includes('open about page') || message.includes('go to about page') ||
message.includes('about')) {

        window.location.href = "/aboutus";

        speech.text = "Opening About page " + username;

    }else if (message.includes('open my profile') || message.includes('my account') || message.includes('go to my
account')|| message.includes('go to my profile')) {

        window.location.href = "/profile";

        speech.text = "Opening your profile page " + username;

    }

     else if (message.includes('open my orders') || message.includes('my order') || message.includes('go to my
orders')) {

        window.location.href = "/user_orders";

        speech.text = "Opening your orders page " + username;

    } else if (message.includes('back to home')  || message.includes('back to home page')) {

        window.location.href = "/home";

        speech.text = "Opening HOME page " + username;

    } else if (message.includes('open services page') || message.includes('go to services') ||
message.includes('services')) {

        window.location.href = "/services_view";

        speech.text = "Opening Services page " + username;

    }else if (message.includes('open feedback page') || message.includes('go to feedback') ||
message.includes('feedback')) {

        window.location.href = "/feedback_view";

        speech.text = "Opening feedbback page " + username;

    }  else if (message.includes('log out') || message.includes('sign out')) {

        window.location.href = "/logout";

        speech.text = "thank you for using Smart-mart i wish you to come again " + username;

    }

    else if (message.includes('open maps page') || message.includes('continue shopping') || message.includes('go to
maps')|| message.includes('continue shoping')) {

        window.location.href = "/maps";

        speech.text = "Opening maps page " + username;

    } else if (message.includes('back to previous page')|| message.includes('back')) {

        history.back();

        speech.text = "Going back to the previous page";

    }

    else if (message.includes('help')|| message.includes('help me')|| message.includes('how can use this')||
message.includes('not understanding')) {

        speech.text = "To order products in Smart-mart, first register, then login. Go to maps and select any one of
the nearby shops. Then select categories and products. Download the invoice and proceed to payment. You can choose
online, offline, or cash on delivery. Finally, place your order.";

    }

     else if (message.includes('time')) {

        const time = new Date().toLocaleString(undefined, { hour: "numeric", minute: "numeric" });
```

```
        speech.text = time;
    } else if (message.includes('date')) {
        const date = new Date().toLocaleString(undefined, { month: "short", day: "numeric" });
        speech.text = date;
    } else if (message.includes('stop assistant')) {
        assistantActive = false;
        recognition.stop();
        speech.text = "Assistant stopped";
        window.location.href = "/home";
    } else if (message.includes('open category')) {
        const parts = message.split('open category ')[1].trim().split(' ');
        const shopId = parts[0];
        const categoryName = parts.slice(1).join(' ');
        if (shopId && categoryName) {
            window.location.href = `/display_categories/?shopid=${shopId}&name=${categoryName}`;
            speech.text = `Opening category ${categoryName} in shop ${shopId}`;
        } else {
            speech.text = "Please specify a shop ID and category name";
        }
    } else if (message.includes('open product')) {
        const productName = message.split('open product ')[1].trim();
        if (productName) {
            window.location.href = `/display_products/?name=${productName}`;
            speech.text = `Opening product ${productName}`;
        } else {
            speech.text = "Please specify a product name";
        }
    } else {
        speech.text = "I did not understand what you said, please try again";
    }
    speech.volume = 1;
    speech.pitch = 1;
    speech.rate = 1;
    window.speechSynthesis.speak(speech);
}
window.addEventListener('load', () => {
    const welcomeSpoken = sessionStorage.getItem('welcome_spoken') = 'false';
    const errorMessage = sessionStorage.getItem('error_message');
    if (!welcomeSpoken) {
        speak("Activating Smart-mart Assistant");
```

```
        speak("Going online");
        wishMe();
        sessionStorage.setItem('welcome_spoken', 'true');
    }
    else{
        speak("how can i help you, Boss");
    }
    if (errorMessage) {
        speak(errorMessage);
        sessionStorage.removeItem('error_message');
    }
    recognition.start();
});
btn.addEventListener('click', () => {
    recognition.start();
});
```

# TESTING

# 7.TESTING

## 7.1. INTRODUCTION

Software Testing is a method to check whether the actual software product matches expected requirements and to ensure that the software product is Defect free. It involves the execution of software/system components using manual or automated tools to evaluate one or more properties of interest. The purpose of software testing is to identify errors, gaps, or missing requirements in contrast to actual requirements.. Properly tested software product ensures reliability, security, and high performance which further results in time-saving, cost-effectiveness, and customer satisfaction. Testing is important because software bugs could be expensive or even dangerous. Software bugs can potentially cause monetary and human loss, and history is full of such examples. i)Starbucks was forced to close about 60 percent of stores in the U.S. and Canada due to software failure in its POS system. At one point, the store served coffee for free as they were unable to process the transaction. ii) Some of Amazon's third-party retailers saw their product price reduced to 1p due to a software glitch. They were left with heavy losses. iii) Vulnerability in Windows 10. This bug enables users to escape from security sandboxes through a flaw in the win32k system. iv) In April of 1999, a software bug caused the failure of a $1.2 billion military satellite launch, the costliest accident in history.

## 7.2 TYPES OF SOFTWARE TESTING

### 7.2.1 BLACK BOX TESTING

- **Definition:** Testing where the tester evaluates the software's functionality without any knowledge of the internal code structure, implementation details, or internal paths.
- **Purpose:** To check if the software performs as expected based on the requirements and specifications.
- **Focus:** Input-output behavior, user interface, and system functionality.

### 7.2.2 GRAY BOX TESTING

- **Definition:** Testing where the tester has partial knowledge of the internal workings of the software, combining aspects of both black box and white box testing.

- **Purpose:** To enhance the quality of black box testing by utilizing knowledge of the internal structure to design better test cases and to identify hidden issues.
- **Focus:** Internal processes, data flow, and the interaction between internal components and the external user interface.

### 7.2.3 UNIT TESTING

- **Definition:** Tests individual components or functions of a software.
- **Purpose:** To ensure that each unit of the software performs as expected.
- **Example Tools:** J Unit, N Unit.

### 7.2.4 INTEGRATION TESTING

- **Definition:** Tests the combination of individual units and how they work together.
- **Purpose:** To identify issues in the interaction between integrated units.
- **Example Tools:** J Unit (for testing modules), Postman (for API testing).

### 7.2.5 SYSTEM TESTING

- **Definition:** Tests the complete, integrated system.
- **Purpose:** To validate the system's compliance with the specified requirements.
- **Example Tools:** Selenium, QTP.

### 7.2.6 PERFORMANCE TESTING

- **Definition:** Tests the software's performance under specific conditions.
- **Purpose:** To check the system's speed, responsiveness, and stability.
- **Types:**
    - **Load Testing:** Tests performance under expected load.
    - **Stress Testing:** Tests performance under extreme conditions.
    - **Scalability Testing:** Tests the software's ability to scale up.
    - **Volume Testing:** Tests performance with a large volume of data.
- **Example Tools:** J Meter, Load Runner.

### 7.2.7 SECURITY TESTING

- **Definition:** Tests the software for vulnerabilities.
- **Purpose:** To identify and fix security issues.
- **Example Tools:** OWASP ZAP, Burp Suite.

### 7.2.8 USABILITY TESTING

- **Definition:** Tests the software's user-friendliness.
- **Purpose:** To ensure the software is easy to use and meets user expectations.
- **Example Tools:** Usability Hub, User Testing.

### 7.2.9 COMPATIBILITY TESTING

- **Definition:** Tests the software's compatibility with different environments.
- **Purpose:** To ensure the software works well across various devices, browsers, and operating systems.
- **Example Tools:** Browser Stack, Sauce Labs.

# RESULTS

# 8.RESULT

## LOGIN PAGE

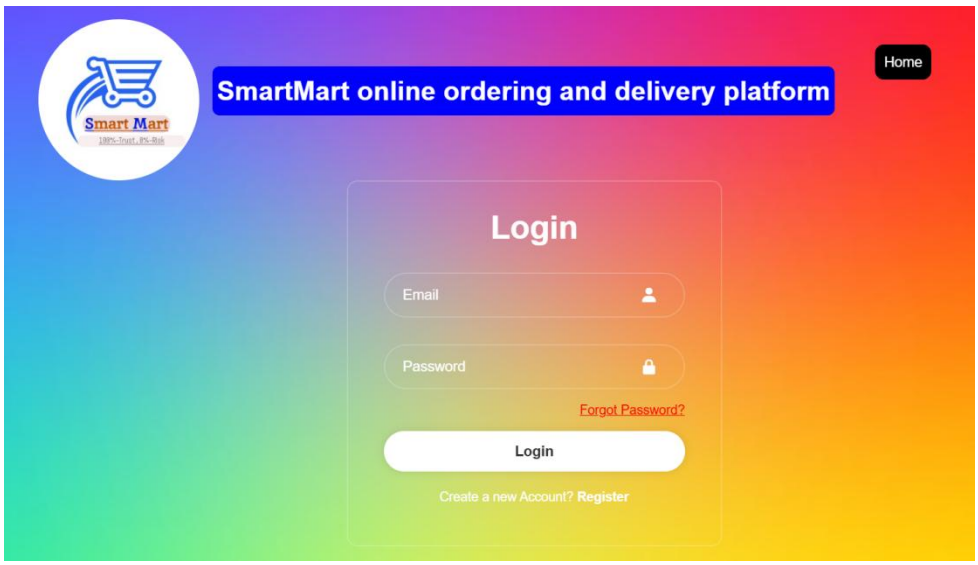


Fig 8.1 Login page

## REGISTRATION PAGE

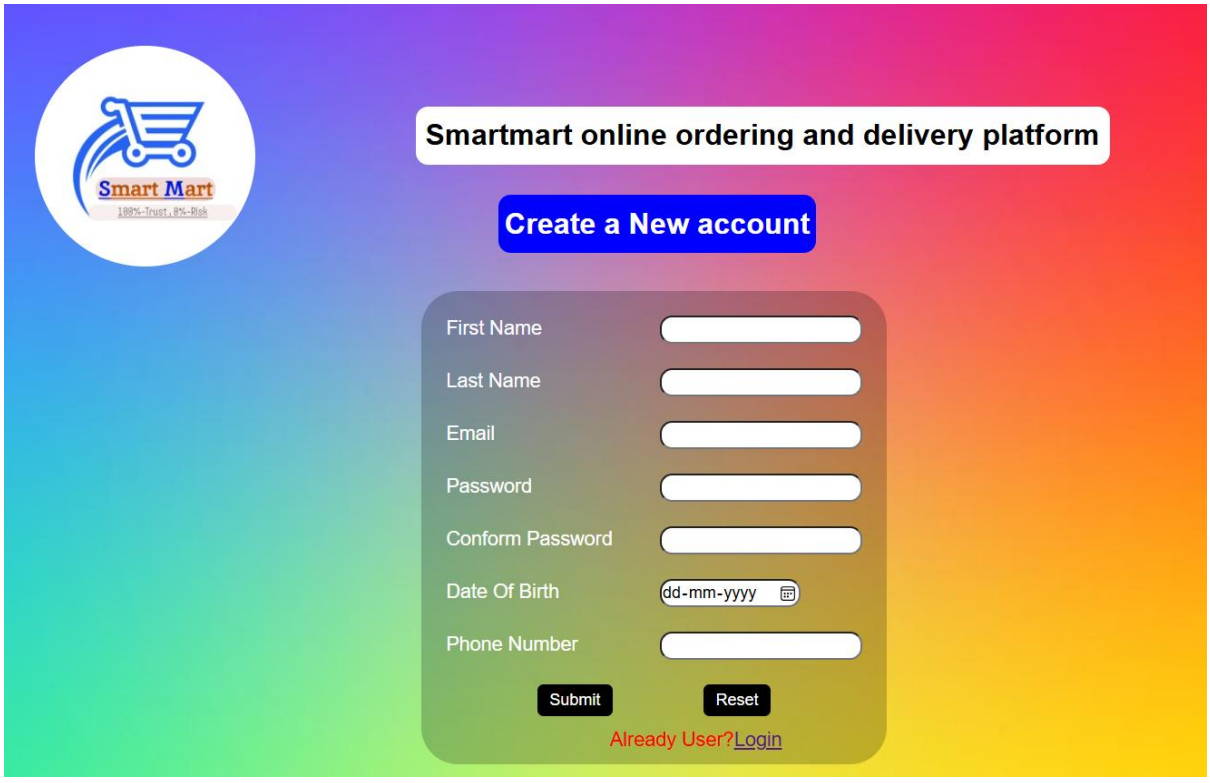


Fig 8.2 sign-up page

## HOME PAGE

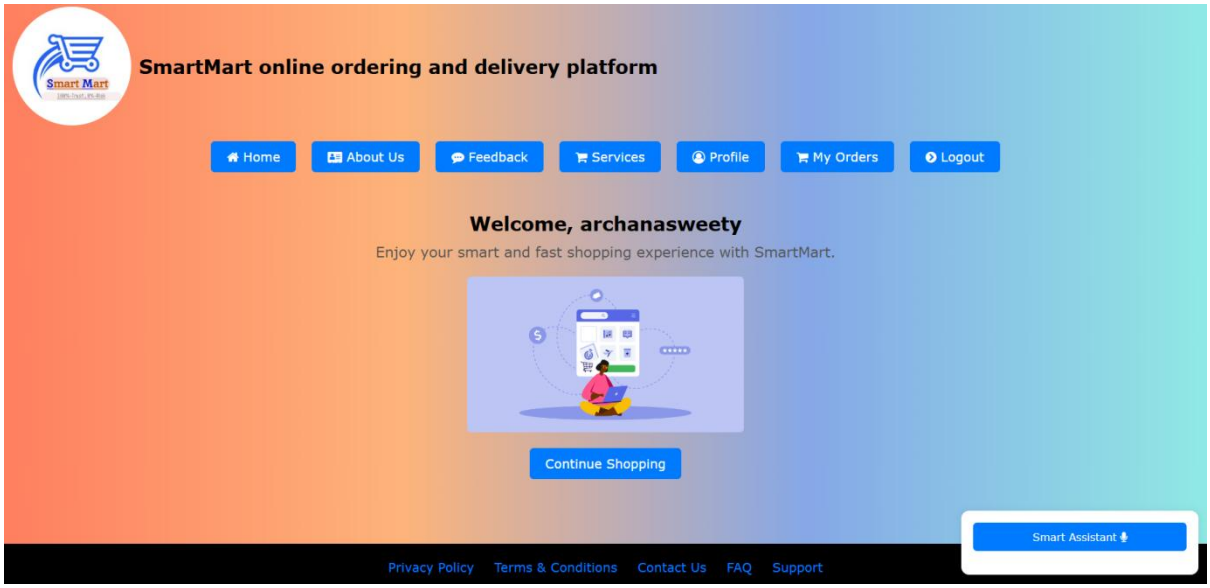


Fig 8.3 Home page

## MAPS PAGE

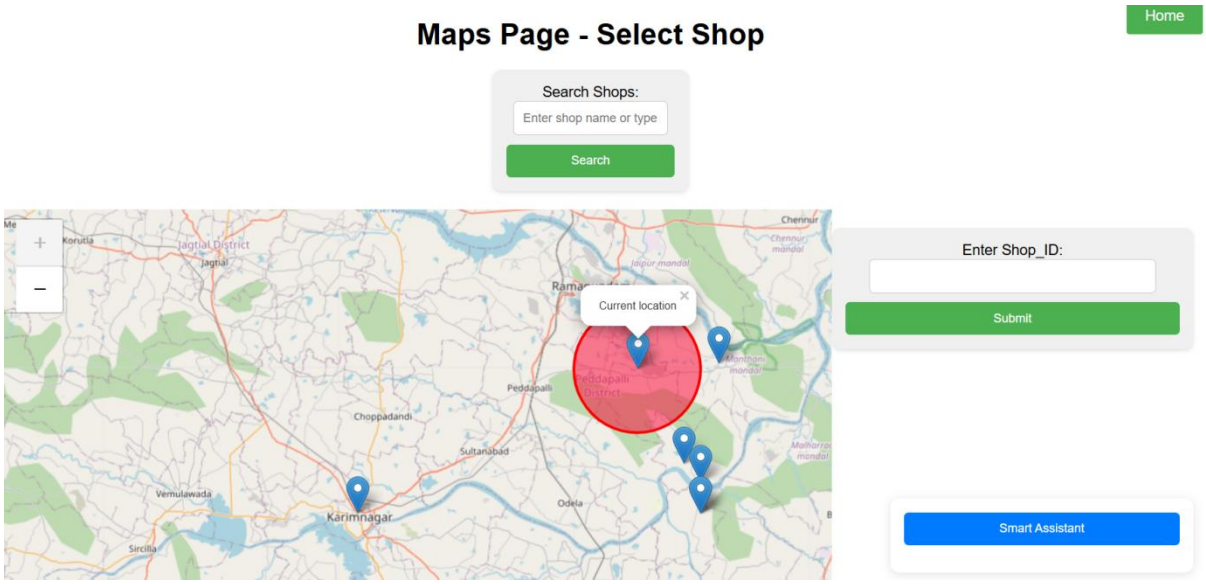


Fig 8.4 Map page

## SHOP CATEGORIES PAGE

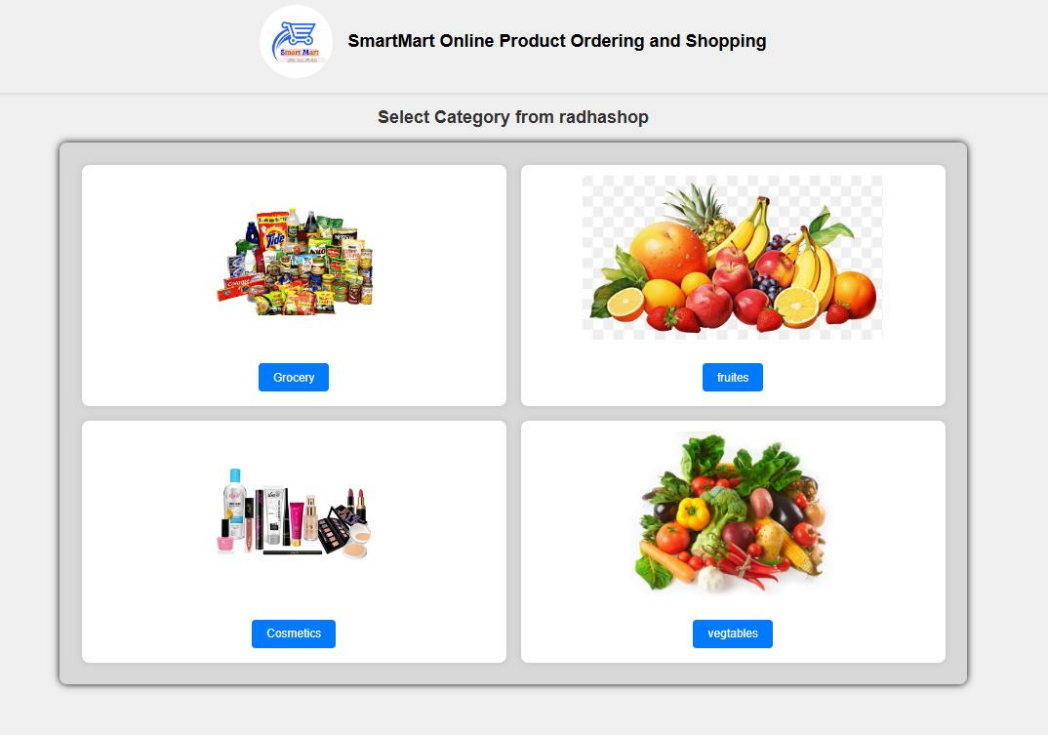


Fig 8.5 Category page

## SHOP PRODUCTS PAGE

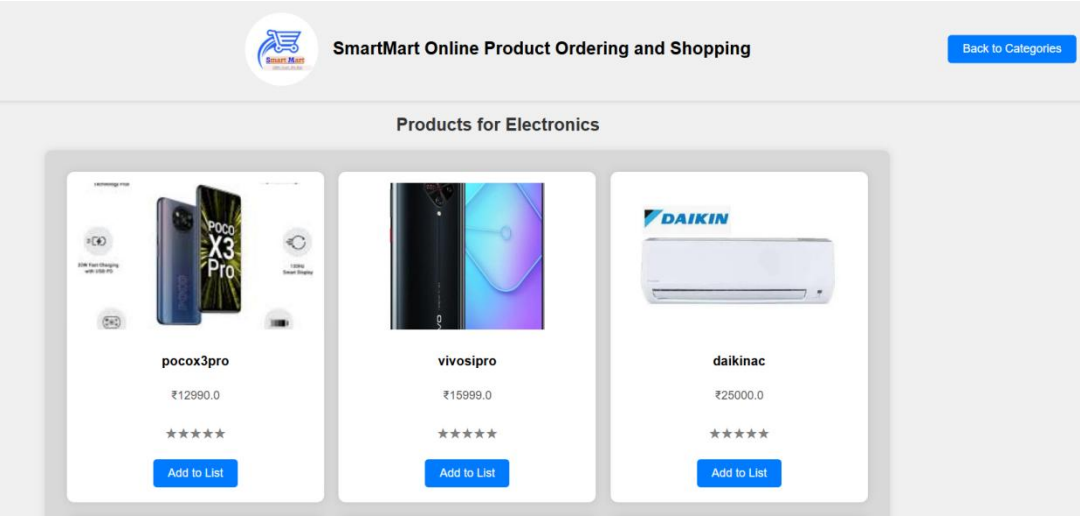


Fig 8.6 Products page

## CHECKOUT/CART PAGE

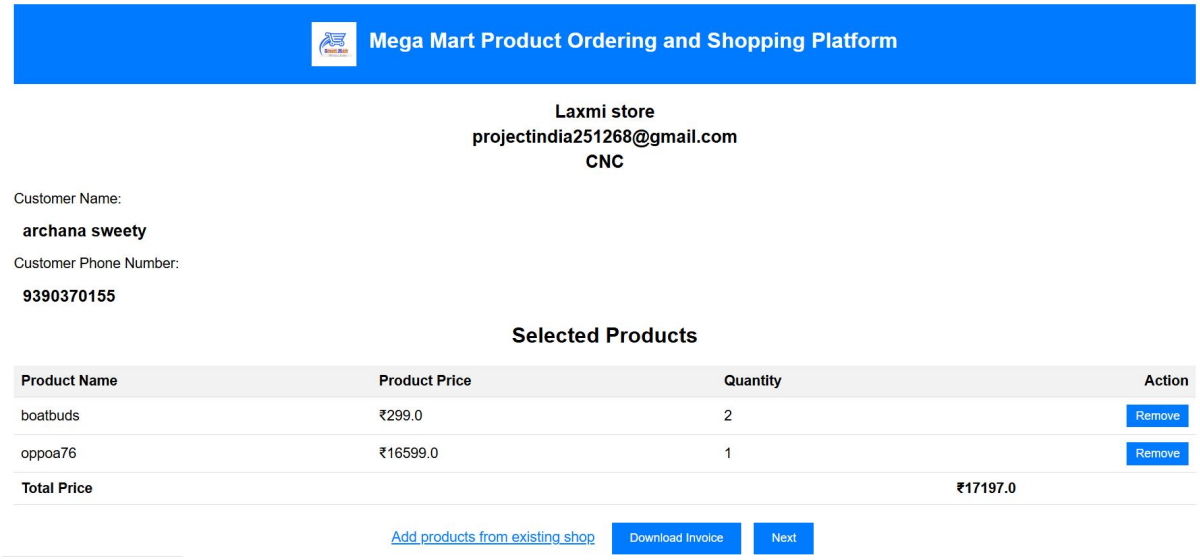


Fig 8.7 Checkout page

## INVOICE PDF

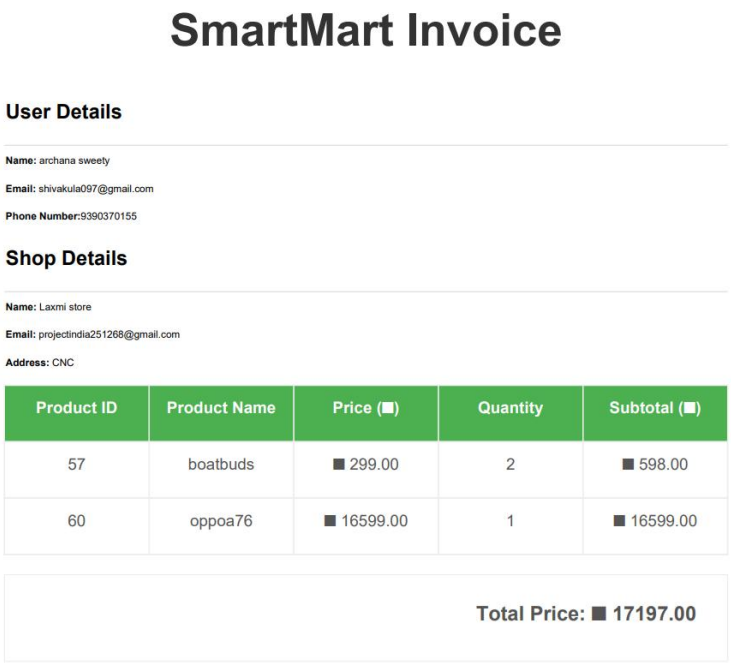


Fig 8.8 Invoice page

## ORDER PLACE PAGE

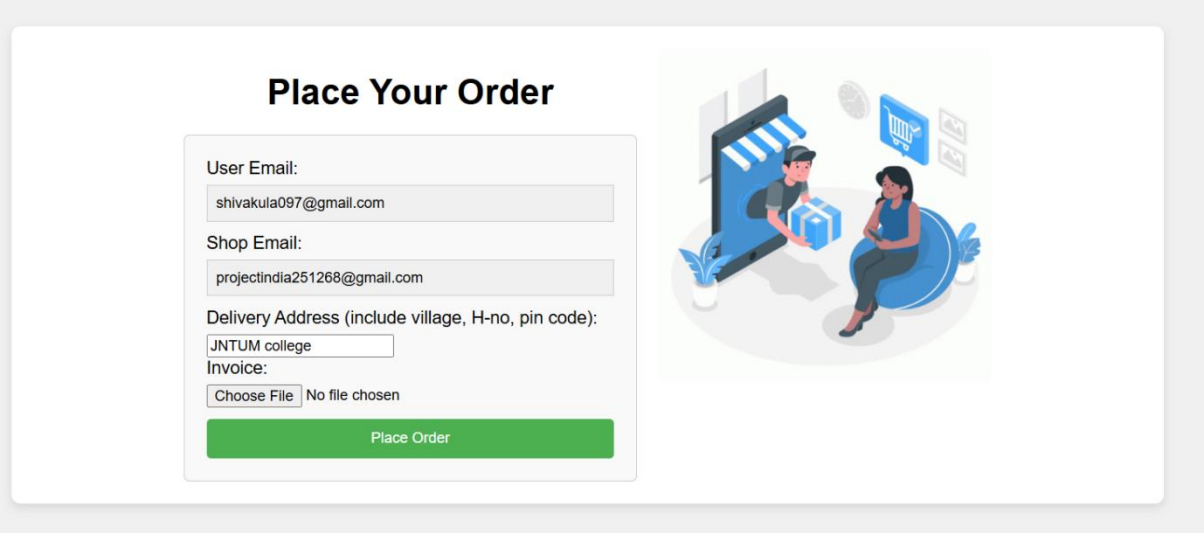


Fig 8.9 Order place page

## PAYMENTS PAGE

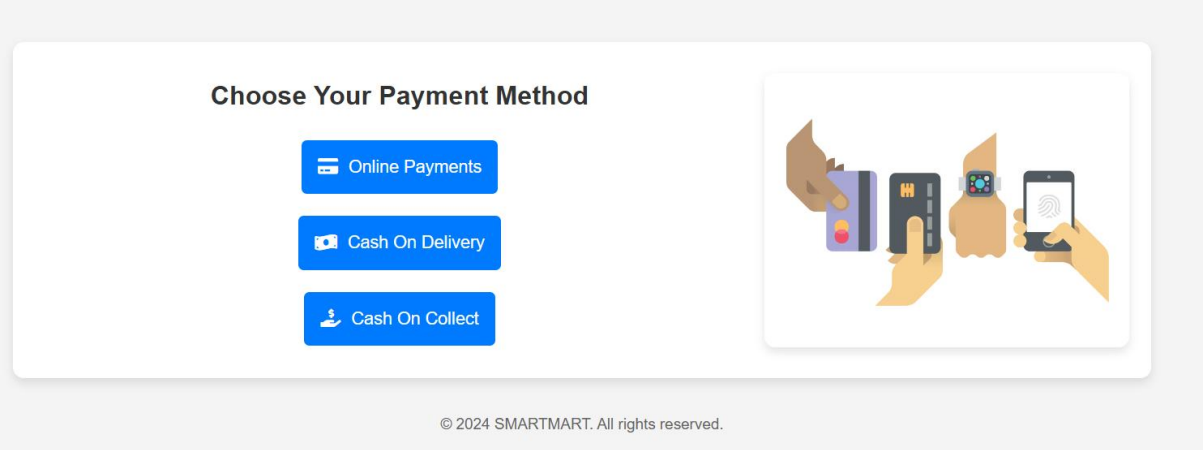


Fig 8.10 payments page

## ORDER SUCCESS PAGE

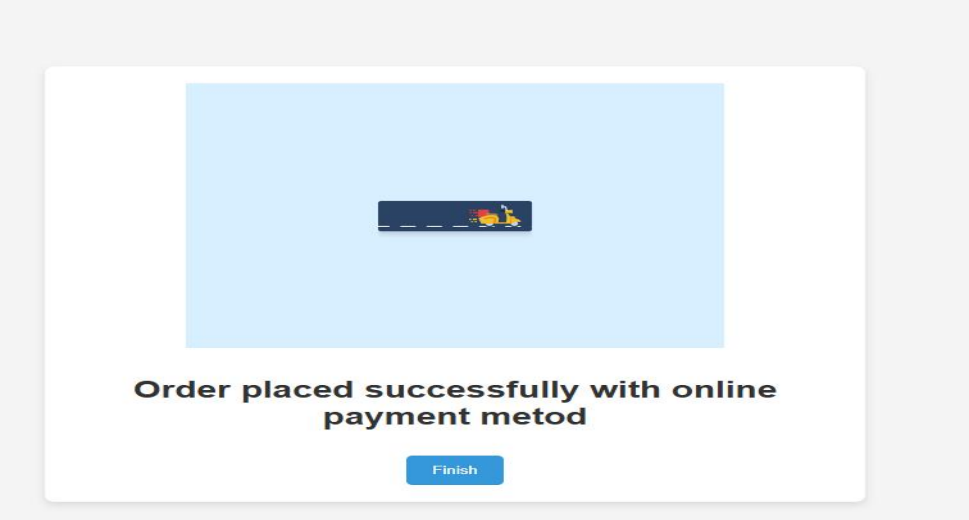


Fig 8.11 order success page

## MY_ORDERS PAGE

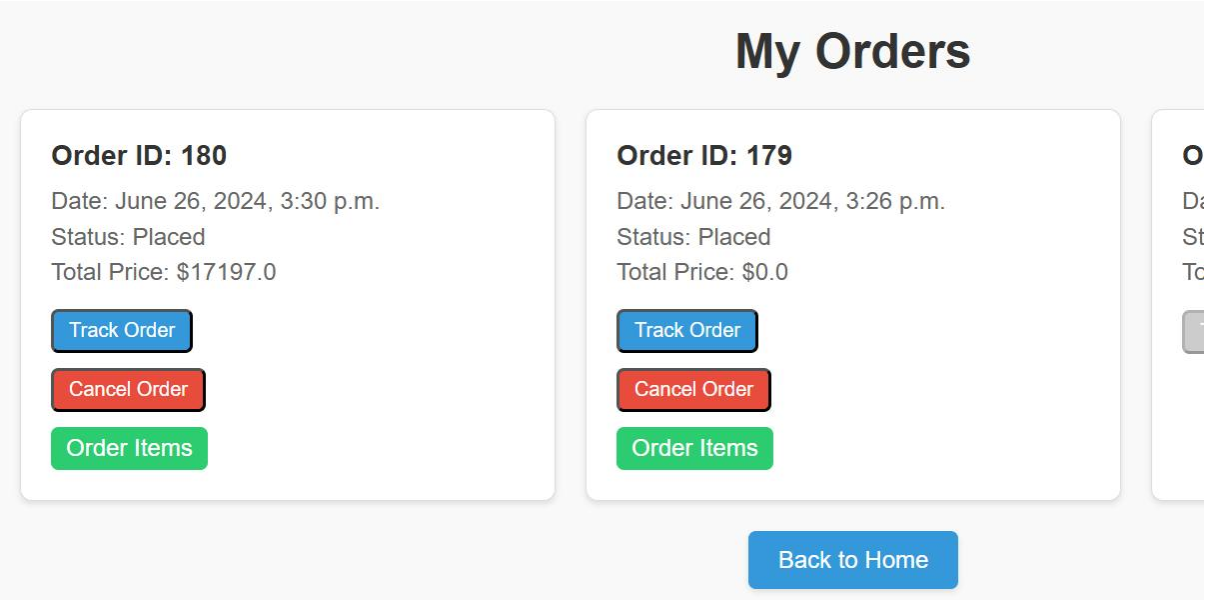


Fig 8.12 User orders page

## TRACK ORDERS PAGE

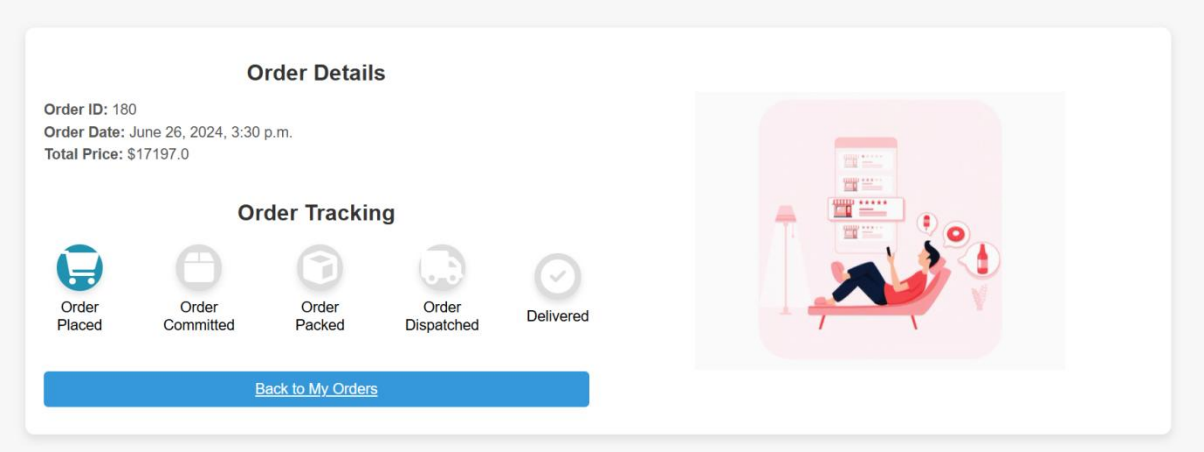


Fig 8.13 Track order page

## SMART MART VOICE ASSISTANT

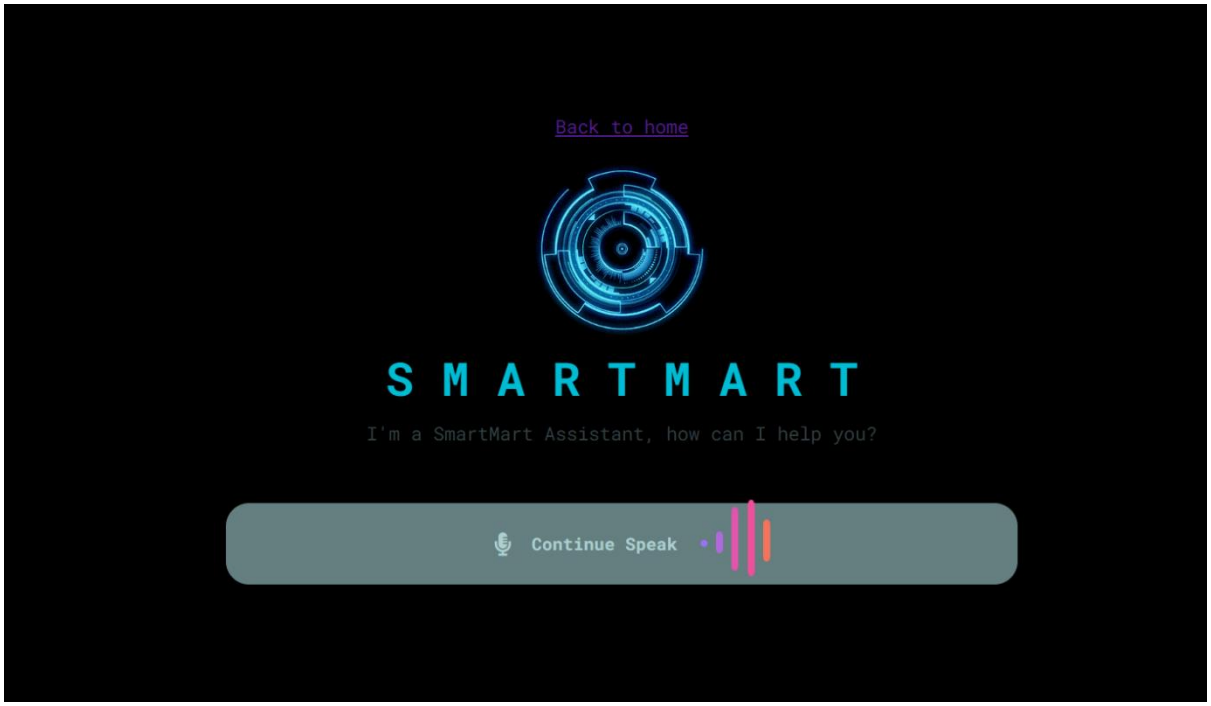


Fig 8.14 Assistant page

# CONCLUSION

# 9.CONCLUSION

**SMART-MART** is designed to boost local businesses by making it easy for customers to shop online from nearby stores. It's built to be user-friendly, secure for payments, and efficient in managing orders. By focusing on scalability, safety, and following rules, Smart-mart aims to help local businesses grow and improve delivery services. It's set to lead in online shopping, making local economies stronger and more accessible to everyone.

Mainly the project developed on client and server architecture. The online shopper should select the supermarket or store within the specified geographical range. The website provides the products of the selected supermarket and displays the products in the form of GUI which may attract the customers, now the customer selects the necessary products and make bill list.

The soft copy of the bill receipt store at server side and client side now the smart-mart web-server sends the attachment of bill receipt soft-copy to the previously select supermarket the supermarket will pack the items based on the bill receipt items, the payment process done on user selected method. Considering driving factors to Smart-mart online shopping people are more inclined to shop online just because they receive high discounts and delivery in 24hours, product pricing and also variety in product range. More number of people have preferred to shop online as its time saving and home delivered. Lastly, the type of payment mode preferred by response is somewhat surprising because more people have preferred cash on delivery whereas, UPI/net-banking is preferred least and credit/debit card and mobile wallet are preferred moderately and also cash on collect method was used. Cash on collect is a new method used in smart-mart shopping the user should pay the bill and collect the packed items from the supermarket.

From the results we have concluded that the most influencing and attractive five factors particularly the security concerns are very important while shopping online. Last but not least after analyzing we have found that user can select the supermarket to place the orders within the geographical range, low price, discount, deliveries the products within the 24 hours, product pricing, and quality of product and information are also considered to be important factors.

# BIBLIOGRAPHY

# REFERENCES


- "Django for Beginners: Build websites with Python and Django" by William S. Vincent
- "Django for API's: Build web API's with Python and Django" by William S. Vincent
- Django Software Foundation, "Django Documentation," Django Project.
- Cool Text for creating stylish images and buttons for the Smart Mart interface.
  - Cool Text, "Cool Text: Logo and Graphics Generator," Cool Text.
- Smart Draw for creating professional diagrams, including flowcharts, UML diagrams, and network diagrams for Smart Mart.
  - Smart Draw, "Smart Draw: Diagrams and Charts," Smart Draw.
- Best practices for implementing secure payment processing and integrating different payment methods in Smart Mart.
  - https://dashboard.razorpay.com/?screen=sign_in&next= app&country_code=IN